

11 – Manipulando y validando textos

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express de la clase anterior 06/05

- ¿Que es un archivo y para que podemos usarlo?
- ¿Como leemos un archivo?
- ¿Como lo escribimos?

Trabajando con datos

Manipulando y validando strings

Extrayendo información prolijamente

Cuando trabajamos con archivos de texto o simplemente con textos podemos encontrarnos en la necesidad de manipular de diferentes maneras el texto para que sea útil a nuestro programa.

Tenemos el siguiente desafío, convertir el siguiente texto en una salida ordenada nuestro gusto:

“ AnA ;8;7;9”

“ JuAn ;4;5;3”

“ LucIA ;10;9;10”

Salida esperada:

Alumno 'Ana' - Notas: 8 - 7 - 9

Alumno 'Juan' - Notas: 4 - 5 - 3

Alumno 'Lucía' - Notas: 10 - 9 - 10

Función split

La función **SPLIT** nos permite convertir una cadena de texto en una lista separándolo por delimitador.

Su sintaxis es la siguiente: **STRING.split(DELIMITADOR, MAX_SPLIT=-1)**

Max split por defecto es -1 lo cual es infinito.

Ejemplo de uso:

```
>>> "Buenos dias estrellitas".split(" ")  
['Buenos', 'dias', 'estrellitas']
```

Volviendo a nuestro problema: Extraigamos el nombre de esta línea: " AnA ;8;7;9"

Podemos identificar que el delimitador para resolver nuestro ejercicio es ";" por lo cual vamos a ejecutar la siguiente sentencia:

```
>>> " AnA ;8;7;9".split(";")  
[' AnA ', '8', '7', '9']
```

Función strip

La función **strip** nos permite eliminar los caracteres vacíos e invisibles de una cadena de texto al principio y al final de misma. No en el medio.

Su sintaxis es la siguiente: **STRING.strip(characters=None)**

Characters indica cuales son los caracteres que se deben eliminar de los extremos, normalmente se deja vacío.

Ejemplo de uso:

```
>>> " Hola Mundo ".strip() # "Hola Mundo"
>>> "$Hola Mundo$".strip("$")
```

Volviendo a nuestro problema: Extrajimos el nombre pero con sus espacios " AnA "
Con un simple trim sobre el nombre podemos dejarlo completamente limpio.

```
>>> " AnA ".strip()
'AnA'
```

Función capitalize

La función **capitalize** transforma el texto para que solo el primer carácter sea mayúscula y el resto minúscula, esto nos permite la normalización de palabras.

Su sintaxis es la siguiente: **STRING.capitalize()**

Ejemplo de uso:

```
>>> "HOLA".capitalize() # "Hola"
>>> " HOLA".capitalize() # ¿Que pasa con esto?
```

Asociada a **capitalize** también existen las funciones **lower** que pasa todo nuestro texto a minúscula. Y la función **upper** que pasa todo el texto a mayúscula.

Volviendo a nuestro problema: Extrajimos el nombre y limpiamos sus espacios "AnA" Ahora podemos capitalizarlo, y TARAN ya tenemos nuestro nombre normalizado.

```
>>> 'AnA'.capitalize()
'Ana'
```



Función join

La función **join** nos permite unir listas de strings mediante un carácter y convertirlo en un string.

Su sintaxis es la siguiente: **STRING.join(lista)**

Ejemplo de uso:

```
>>> " ".join(["Soy", "un", "programador"]) # Soy un programador
>>> "\n".join(["Soy", "un", "programador"])
```

Volviendo a nuestro problema: Todavía nos faltan la representación de nuestras notas.
a partir del split podemos obtener la lista de notas y con join podemos unir las simplemente con " - ".

Ahora podemos capitalizarlo, y TARAN ya tenemos nuestro nombre normalizado.

```
>>> " - ".join(['8', '7', '9'])
'8 - 7 - 9'
```


Ejercicio manipulación de datos

Ejercicio 1 - Normalizar datos de alumnos

Dada la siguiente lista:

```
lineas = [  
  "AnA ;8;7;9",  
  "JuAn ;4;5;3",  
  "LucIA ;10;9;10"  
]
```

Mostrar cada alumno con este formato:

Alumno 'Ana' - Notas: 8 - 7 - 9
Alumno 'Juan' - Notas: 4 - 5 - 3
Alumno 'Lucía' - Notas: 10 - 9 - 10

Ejercicio 2 - Crear mensajes desde datos

Dada la siguiente lista:

```
usuarios = [  
  "ana,programacion",  
  "juan,matematica",  
  "lucia,fisica"  
]
```

Mostrar teniendo en cuenta las mayúsculas:

Hola Ana, estás inscripto/a en Programacion.
Hola Juan, estás inscripto/a en Matematica.
Hola Lucia, estás inscripto/a en Fisica.

Ejercicio 3 - Función del camello

Escribir una función que convierta una frase en camel case.

"pedro MARTINEZ sEgundo" -> "pedro Martinez Segundo"



Slicing de listas

Python ofrece un mecanismo para manipular el acceso de diversas maneras al contenido de las listas.

SPOILER: También sirve para strings

- Sintaxis: `[inicio:fin:paso]`.
 - Inicio: Índice donde comienza (default 0).
 - Fin: Índice donde termina (no incluido) (default longitud de secuencia).
 - Paso: Tamaño del salto entre elementos (default 1).
- Índices Negativos: Permiten acceder a elementos desde el final, útil para obtener los últimos elementos sin saber la longitud exacta.
- Inversión: Usar un paso de `-1` (`[::-1]`) invierte la secuencia.
- Flexibilidad: Si se omiten parámetros, Python asume los valores por defecto, ej: `[:]` copia toda la lista

Slicing de listas

Python ofrece un mecanismo para manipular el acceso de diversas maneras al contenido de las listas.

SPOILER: También sirve para strings

Casos Prácticos y Ejemplos

En los siguientes ejemplos usaremos la lista: `letras = ['a', 'b', 'c', 'd', 'e']`.

Caso de uso [1] , [2] , [3]	Código de ejemplo	Resultado
Extraer un rango	<code>letras[1:4]</code>	<code>['b', 'c', 'd']</code>
Desde el inicio	<code>letras[:3]</code>	<code>['a', 'b', 'c']</code>
Hasta el final	<code>letras[2:]</code>	<code>['c', 'd', 'e']</code>
Copiar lista completa	<code>letras[:]</code>	<code>['a', 'b', 'c', 'd', 'e']</code>
Con saltos (paso)	<code>letras[::2]</code>	<code>['a', 'c', 'e']</code>
Lista invertida	<code>letras[::-1]</code>	<code>['e', 'd', 'c', 'b', 'a']</code>
Índices negativos	<code>letras[-3:-1]</code>	<code>['c', 'd']</code>

Practicando slicing

Ejercicio 4 - Primeros y últimos elementos

Dada la lista:

```
numeros = [10, 20, 30, 40, 50, 60]
```

Mostrar:

- Los primeros 3 elementos
- Los últimos 2 elementos
- Cada dos elementos
- Al revés

Ejercicio 5 - Recortar una palabra

Dada la palabra:

```
palabra = "programacion"
```

Mostrar:

- Las primeras 5 letras.
- Las últimas 4 letras.
- La palabra invertida.

Ejercicio 6 - Dando vuelta las palabras

Definir una función que reciba una oración y devuelva la misma oración con el orden de las palabras invertido, sin usar split.

Ejemplo: `invertir_palabras("hola mundo python")`

Debe devolver: `python mundo hola`

Validación de datos

¿Porque validar datos?

Cuando trabajamos con datos ingresados por un usuario, leídos desde un archivo o extraídos de un string, no podemos asumir que siempre tendrán el formato esperado.

Validar los datos nos permite revisar mínimamente su contenido antes de usarlos.

Por ejemplo:

- Si esperamos un número y recibimos texto, el programa puede fallar al convertirlo.
- Si esperamos datos separados por “,” y la coma no está, no podremos separar correctamente.
- Si esperamos un nombre y recibimos un número, quizás el programa no falle, pero el dato será inconsistente.

Validar ayuda a construir programas más seguros, claros y confiables.

Las herramientas de validación son muchas, estás son algunas para tener ideas disparadoras.

Validando números en la entrada

La función `isnumeric` no permite determinar si un texto es o no un número:

```
"1996".isnumeric()
# True

"Mil novecientos noventa y nueve".isnumeric()
# False
```

NOTA: Python no ofrece ningún mecanismo para validar floats sobre strings.

A partir de esto con un simple `if` podemos resguardar nuestro programa:

```
fecha_nacimiento = 0
fecha_nacimiento_raw = input("Ingrese su fecha de nacimiento [yyyy]: ")
if fecha_nacimiento_raw.isnumeric():
    fecha_nacimiento = int(fecha_nacimiento_raw)
    print("Fecha de nacimiento asignada exitosamente")
else:
    print("Fecha de nacimiento invalida")
    exit() # Usamos exit para salir inmediatamente dle programa
```

Validando tamaño de la entrada

En ocasiones no va a interesar que el tamaño de una entrada sea el que necesitamos, por ejemplo, para el caso del año, si lo vamos a utilizar matemáticamente, estamos obligados a tener los 4 caracteres.

La opción más simple para esto es len con un comparador.

```
len("1996") == 4 # True
```

```
len(" 1996") == 4 # ¿ Que pasa acá?
```

A partir de esto con un simple if podemos resguardar nuestro programa:

```
fecha_nacimiento = 0
fecha_nacimiento_raw = input("Ingrese su fecha de nacimiento [yyyy]: ")
if fecha_nacimiento_raw.isnumeric() and len(fecha_nacimiento_raw.strip())
== 4:
    fecha_nacimiento = int(fecha_nacimiento_raw)
    print("Fecha de nacimiento asignada exitosamente")
else:
    print("Fecha de nacimiento invalida, se esperaban 4 digitos, EJ:
1992")
    exit() # Usamos exit para salir inmediatamente del programa
```


Validando ingreso de palabras

Python provee `isalpha` para validar el ingreso de palabras, no soporta texto con espacios por ejemplo, tampoco caracteres especiales ni números.

```
"Maria".isalpha() # True  
"Maria Cecilia".isalpha() # False
```

Nos es muy útil para pedir nombres o cuando necesitamos respuesta de Si y No.

NOTA: también existe `isalnum()` valida que un texto sea alfanumérico, es decir caracteres de A-Z y números sin espacios ni símbolos extraños.

```
primer_nombre = ""  
primer_apellido = ""  
primer_nombre_raw = input("Ingrese su primer nombre: ")  
primer_apellido_raw = input("Ingrese su primer apellido: ")  
if primer_nombre_raw.strip().isalpha() and  
primer_apellido_raw.strip().isalpha():  
    primer_nombre = primer_nombre_raw  
    primer_apellido = primer_apellido_raw  
    print("Nombre y apellido registrado correctamente")  
else:  
    print("Error nombre o apellido invalido")
```

Validando caracteres esperados

Cuando esperamos el texto en cierto formato, o por ejemplo separado por “,” podemos utilizar funciones como count o find.

count cuenta la cantidad de apariciones, para datos estructurados donde sabemos cuántos columnas esperamos nos viene muy bien.

```
"Pedro,Martinez,19,2004".count(",") == 3 # True
```

```
nombre_apellido = input("Ingrese nombre,apellido: ")
if nombre_apellido.count(",") == 1:
    nombre, apellido = nombre_apellido.split(",")
    print("Validación correcta")
else:
    print("Error se debe ingresar nombre,apellido")
```

Validaciones de rango

Si sabemos los rangos en los que un valor de entrada debe operar es importante validarlo, para que un usuario no pueda poner un valor por fuera de esos rangos.

```
UMBRAL_MINIMO = 10
UMBRAL_MAXIMO = 20
dato_entrada = int(input("Ingresar dato"))
if UMBRAL_MINIMO <= dato_entrada <= UMBRAL_MAXIMO:
    print("Dato valor")
```

Ejercicios

Ejercicio 7 - Cargar nota de parcial

Pedir al usuario una nota de parcial.

Validar que:

- Sea numérica.
- Esté entre 0 y 10.

Si es válida, mostrar

Ejercicio 8 - Lista de productos separada por coma

Pedir al usuario que ingrese productos separados por coma.

Validar que se ingresó más de dos productos, y que ninguno quedó vacío.

Ejercicio 9 - Validar patente simple

Pedir al usuario una patente con el siguiente formato:
Dos letra + tres números + dos letras (AB123CD)

Validar que:

- No tenga espacios.
- Tenga 7 caracteres.
- Contenga solo letras y números.
- El formato

Ejercicio 10 - Validar float

Pedir al usuario un número flotante implementar un mecanismo para validar que el dato ingresado es correcto.

Momento
Github

PDFS y TAREAS



Todo el material en <https://github.com/Ymil/unrn-programacion-1/>

FIN

Volvemos el miércoles a las 17